

DD_ASR-01 - Technical Trouble - Developer Implementation Guide

Version: 1.0 Status: Draft for review Bundle: 10_other Audience: mid-level backend developers, ticketing developers, QA
Parent DD: DD_ASR-01 - Technical Trouble

1. How To Read This Guide

This guide follows the flow as it executes.

Technical trouble is a TCK ticket flow. Do not create a separate ASR ticket table or lifecycle engine. Create a TECHNICAL_SUPPORT ticket in TCK, then apply ASR diagnostic, outage, WO, and compensation rules.

2. Step 1 - Seed Technical Trouble Categories

Seed TCK category rows for technical support.

Example:

```
insert into ticket_category_config (
  category_id,
  operator_code,
  type_code,
  category_code,
  display_name,
  default_priority,
  default_queue_code,
  default_sla_policy,
  wo_allowed,
  default_wo_kind,
  status
) values
('tcat-nointernet', 'WIK', 'TECHNICAL_SUPPORT', 'NO_INTERNET', 'No internet', 'P2', 'TECH_SUPPORT_L2', 'TECH_P2_8H', true,
'tcat-slowspeed', 'WIK', 'TECHNICAL_SUPPORT', 'SLOW_SPEED', 'Slow speed', 'P3', 'TECH_SUPPORT_L1', 'TECH_P3_24H', true,
'tcat-novoice', 'WIK', 'TECHNICAL_SUPPORT', 'NO_VOICE', 'No voice', 'P2', 'TECH_SUPPORT_L2', 'TECH_P2_8H', true, 'SUPPO
'tcat-areaoutage', 'WIK', 'TECHNICAL_SUPPORT', 'AREA_OUTAGE', 'Area outage', 'P1', 'NOC_OUTAGE', 'TECH_P1_4H', false, n
```

The exact TCK category table name should match TCK-01 implementation. The important part is that these are TCK categories, not ASR primary records.

3. Step 2 - Seed ASR Category Behavior

```
insert into asr_trouble_category_config (
  config_id,
  operator_code,
  category_code,
  service_class_code,
  diagnostic_profile_code,
  wo_policy,
  default_wo_kind,
  repeat_fault_window_days,
  credit_policy_code,
  status
) values
('asr-cat-nointernet', 'WIK', 'NO_INTERNET', 'INTERNET', 'INET_BASIC_DIAG', 'REQUIRED_AFTER_DIAG', 'SUPPORT', 30, 'OUTAG
('asr-cat-slowspeed', 'WIK', 'SLOW_SPEED', 'INTERNET', 'INET_SPEED_DIAG', 'REQUIRED_AFTER_DIAG', 'SUPPORT', 30, null, 'A
('asr-cat-novoice', 'WIK', 'NO_VOICE', 'VOICE', 'VOICE_BASIC_DIAG', 'REQUIRED_AFTER_DIAG', 'SUPPORT', 30, 'VOICE_OUTAGE
('asr-cat-areaoutage', 'WIK', 'AREA_OUTAGE', 'ANY', 'OUTAGE_ONLY_DIAG', 'NONE', null, 30, 'OUTAGE_CREDIT_STANDARD', 'ACT
```

Meaning:

- diagnostic_profile_code tells the UI/API which checks are required.
- wo_policy controls whether a support WO can be created.
- credit_policy_code only recommends compensation. BIL still owns credit creation.

4. Step 3 - Seed Diagnostic Profiles

```
insert into asr_diagnostic_profile (
  profile_id,
  operator_code,
  profile_code,
  display_name,
  required_checks_json,
  status
) values (
  'diag-inet-basic',
  'WIK',
  'INET_BASIC_DIAG',
  'Internet Basic Diagnostics',
  '["SUB_ACTIVE", "CUSTOMER_POWER_CHECK", "OUTAGE_CHECK", "ONT_STATUS", "LAST_PROV_COMMAND"]',
  'ACTIVE'
);
```

Suggested checks:

Check code	Purpose	Source
SUB_ACTIVE	Verify subscription is active or allowed support state.	SUB-LM read API
CUSTOMER_POWER_CHECK	Ask customer whether CPE has power.	CSR/customer input
OUTAGE_CHECK	Check known outage or NOC incident.	NOC/outage API or manual link
ONT_STATUS	Check ONT online/offline if API exists.	PROV-INT/NMS read
LAST_PROV_COMMAND	Check last provisioning command failed recently.	PROV-INT

5. Step 4 - Customer Reports Trouble

Backoffice or self-care creates a TCK ticket.

```
POST /api/tickets
Idempotency-Key: care-nointernet-20260603-001
Content-Type: application/json

{
  "operatorCode": "WIK",
  "typeCode": "TECHNICAL_SUPPORT",
  "categoryCode": "NO_INTERNET",
  "sourceChannel": "CALL_CENTER",
  "customerId": "CUS-100045",
  "accountId": "ACC-900045",
  "subscriptionId": "SUB-700045",
  "subject": "Customer reports no internet",
  "description": "Customer says ONT has LOS light blinking after overnight rain.",
  "customerVisible": true
}
```

Expected TCK result:

```
{
  "ticketId": "tck-20260603-000001",
  "ticketNumber": "KE-TCK-2026-000001",
  "statusCode": "OPEN",
  "typeCode": "TECHNICAL_SUPPORT",
  "categoryCode": "NO_INTERNET",
  "priorityCode": "P2",
  "assignedQueueCode": "TECH_SUPPORT_L2"
}
```

After ticket create, ASR should emit `TechnicalTroubleTicketCreated` and project to customer timeline through CUST-INT-01.

6. Step 5 - Load ASR Guidance For The Ticket

Backoffice opens the ticket and calls:

```
GET /api/asr/technical-trouble/tck-20260603-000001/guidance
```

Example response:

```
{
  "ticketId": "tck-20260603-000001",
```

```

"categoryCode": "NO_INTERNET",
"diagnosticProfileCode": "INET_BASIC_DIAG",
"requiredChecks": [
  "SUB_ACTIVE",
  "CUSTOMER_POWER_CHECK",
  "OUTAGE_CHECK",
  "ONT_STATUS",
  "LAST_PROV_COMMAND"
],
"woPolicy": "REQUIRED_AFTER_DIAG",
"defaultWoKind": "SUPPORT",
"repeatFaultWindowDays": 30
}

```

This keeps frontend behavior driven by backend configuration.

7. Step 6 - Run Diagnostics

The CSR or system posts diagnostic results:

```

POST /api/asr/technical-trouble/tck-20260603-000001/diagnostics
Content-Type: application/json

```

```

{
  "profileCode": "INET_BASIC_DIAG",
  "results": [
    {
      "checkCode": "SUB_ACTIVE",
      "resultCode": "PASS",
      "payload": {"subscriptionStatus": "ACTIVE"}
    },
    {
      "checkCode": "CUSTOMER_POWER_CHECK",
      "resultCode": "PASS",
      "payload": {"customerConfirmedPower": true}
    },
    {
      "checkCode": "OUTAGE_CHECK",
      "resultCode": "PASS",
      "payload": {"knownOutage": false}
    },
    {
      "checkCode": "ONT_STATUS",
      "resultCode": "FAIL",
      "payload": {"ontOnline": false, "losLight": true}
    }
  ]
}

```

For each result, insert:

```

insert into asr_ticket_diagnostic_result (
  result_id,
  ticket_id,
  profile_code,
  check_code,
  result_code,
  result_payload_json,
  captured_by_user_id,
  captured_at
) values (
  'diagres-001',
  'tck-20260603-000001',
  'INET_BASIC_DIAG',
  'ONT_STATUS',
  'FAIL',
  '{"ontOnline":false,"losLight":true}',
  'usr-care-001',
  now()
);

```

Then add a TCK timeline event/comment summarizing customer-safe results. Do not expose internal raw diagnostics to self-care.

8. Step 7 - Known Outage Path

If OUTAGE_CHECK finds known outage:

```
POST /api/asr/technical-trouble/tck-20260603-000001/link-outage
Content-Type: application/json
```

```
{
  "outageRef": "OUT-NRB-20260603-01",
  "outageStatus": "OPEN",
  "customerMessage": "We are aware of a service issue in your area and our technical team is working on it."
}
```

Insert:

```
insert into asr_outage_link (
  outage_link_id,
  ticket_id,
  outage_ref,
  outage_status,
  linked_by_user_id,
  linked_at
) values (
  'aol-001',
  'tck-20260603-000001',
  'OUT-NRB-20260603-01',
  'OPEN',
  'usr-noc-001',
  now()
);
```

Then update TCK ticket:

```
status_code = WAITING_INTERNAL
assigned_queue_code = NOC_OUTAGE
```

When outage resolves, ASR or NOC can bulk resolve linked tickets or move them to confirmation.

9. Step 8 - Remote Fix Path

Some technical trouble can be resolved without WO.

Examples:

- customer power-cycles router;
- CSR confirms wrong Wi-Fi network;
- NOC re-applies provisioning profile through PROV-INT force-sync;
- voice registration refreshed.

If remote fix succeeds, call TCK resolve:

```
POST /api/tickets/tck-20260603-000001/resolve
Content-Type: application/json

{
  "resolutionCode": "REMOTE_FIX",
  "resolutionSummary": "Customer rebooted router and service restored. ONT online confirmed.",
  "customerVisibleSummary": "Your service is now restored."
}
```

Ticket moves to RESOLVED or UNDER_REVIEW depending category policy.

10. Step 9 - Create Support Work Order

If diagnostics show field work is needed, call TCK create-WO.

```
POST /api/tickets/tck-20260603-000001/work-orders
Idempotency-Key: tck-20260603-000001-support-wo-001
Content-Type: application/json
```

```
{
  "workOrderKind": "SUPPORT",
  "reasonCode": "NO_INTERNET",
  "priorityCode": "P2",
  "homepassId": "HP-300045",
  "subscriptionId": "SUB-700045",
  "diagnosticSummary": {
    "profileCode": "INET_BASIC_DIAG",
    "failedChecks": ["ONT_STATUS"]
  }
}
```

```

    },
    "customerReportedSymptoms": ["LOS_BLINKING", "NO_CONNECTIVITY"]
}

```

Before calling WO-01, validate:

```

ticket.type_code = TECHNICAL_SUPPORT
category wo_policy != NONE
required diagnostic checks captured
ticket not terminal
no active linked support WO

```

After WO creation succeeds:

```

ticket.status_code = WAITING_WORK_ORDER
ticket.current_wo_id = created WO id

```

Insert TCK link:

```

entity_type = WORK_ORDER
entity_id = created WO id
link_role = CREATED_FROM_TICKET

```

11. Step 10 - Consume WO Finalization

TCK already consumes WO events. ASR adds interpretation for technical trouble.

Example event:

```

{
  "eventType": "WorkOrderFinalized",
  "workOrderId": "wo-20260603-000045",
  "originatingContextType": "TICKET",
  "originatingContextId": "tck-20260603-000001",
  "finalReasonCode": "FIXED_ON_SITE",
  "findings": {
    "rootCause": "DROP_CABLE_CUT",
    "solution": "Replaced drop cable",
    "serviceRestored": true
  }
}

```

ASR action:

WO final reason	Ticket update
FIXED_ON_SITE	UNDER_REVIEW or RESOLVED.
CUSTOMER_NOT_AVAILABLE	WAITING_CUSTOMER.
NETWORK_ESCALATION_REQUIRED	WAITING_INTERNAL, queue NOC_ESCALATION.
NO_FAULT_FOUND	UNDER_REVIEW.
CPE_REPLACED	UNDER_REVIEW or RESOLVED, link equipment if provided.

Always add a ticket timeline entry with customer-safe summary.

12. Step 11 - Detect Repeat Fault

When creating or reopening a technical trouble ticket, search recent resolved tickets:

```

select ticket_id
from ticket
where operator_code = 'WIK'
and subscription_id = 'SUB-700045'
and type_code = 'TECHNICAL_SUPPORT'
and category_code in ('NO_INTERNET', 'INTERMITTENT_CONNECTION')
and status_code in ('RESOLVED', 'CLOSED')
and resolved_at >= now() - interval '30 days';

```

If found:

- set category/subcategory or tag as REPEAT_FAULT;
- raise priority if policy says so;
- route to TECH_SUPPORT_L2 or NOC;

- include prior ticket refs in timeline.

13. Step 12 - Recommend Compensation

If service impact is verified and policy applies:

```
POST /api/asr/technical-trouble/tck-20260603-000001/compensation-recommendations
Content-Type: application/json

{
  "reasonCode": "SERVICE_OUTAGE_VERIFIED",
  "recommendedAmount": 250.00,
  "currency": "KES",
  "notes": "Verified outage lasted 10 hours. Drop cable fault confirmed by W0."
}
```

Insert recommendation:

```
insert into asr_compensation_recommendation (
  recommendation_id,
  ticket_id,
  operator_code,
  reason_code,
  recommended_amount,
  currency_code,
  status,
  created_at
) values (
  'acr-001',
  'tck-20260603-000001',
  'WIK',
  'SERVICE_OUTAGE_VERIFIED',
  250.00,
  'KES',
  'RECOMMENDED',
  now()
);
```

If user chooses to create a formal adjustment request, call BIL-02-ADJ-01:

```
POST /api/adjustment-requests
Content-Type: application/json

{
  "sourceModule": "ASR-01",
  "sourceRefId": "tck-20260603-000001",
  "customerId": "CUS-100045",
  "subscriptionId": "SUB-700045",
  "adjustmentKind": "CREDIT_NOTE",
  "reasonCode": "SERVICE_OUTAGE_VERIFIED",
  "scope": "AMOUNT",
  "amount": 250.00,
  "currency": "KES",
  "notes": "ASR verified service outage from support W0."
}
```

Update recommendation with returned adjustmentRequestId.

14. Step 13 - Resolve Ticket

Resolve through TCK:

```
POST /api/tickets/tck-20260603-000001/resolve
Content-Type: application/json

{
  "resolutionCode": "FIELD_FIX_COMPLETED",
  "resolutionSummary": "Drop cable replaced and service restored.",
  "customerVisibleSummary": "Your internet service has been restored after field repair."
}
```

Expected:

- ticket status becomes RESOLVED or UNDER_REVIEW;
- ASR emits TechnicalTroubleResolved;
- CUST-INT-01 receives timeline projection;

- customer notification is sent if configured.

15. Suggested Worker Commands

These run inside the TCK/ASR worker set and can be consolidated.

```
asr:detect-repeat-faults
asr:sync-outage-status
asr:consume-wo-events
asr:sla-escalation-scan
asr:compensation-policy-scan
```

Recommended schedule:

Command	Frequency
asr:detect-repeat-faults	On ticket create/reopen, plus daily audit.
asr:sync-outage-status	Every 5 minutes if outage source exists.
asr:consume-wo-events	Continuous Kafka consumer.
asr:sla-escalation-scan	Every 5 minutes.
asr:compensation-policy-scan	Daily or manual.

16. Error Codes

Error code	Meaning
ASR_CATEGORY_NOT_TECHNICAL	Ticket category is not valid for ASR-01.
ASR_DIAGNOSTIC_PROFILE_NOT_FOUND	Category has no active diagnostic profile.
ASR_REQUIRED_DIAGNOSTIC_MISSING	WO creation attempted before required checks.
ASR_OUTAGE_ALREADY_LINKED	Ticket already linked to same outage.
ASR_WO_POLICY_BLOCKS_CREATION	Category does not allow WO.
ASR_ACTIVE_WO_ALREADY_EXISTS	Ticket already has active support WO.
ASR_COMPENSATION_NOT_ELIGIBLE	Policy does not allow compensation recommendation.
ASR_ADJUSTMENT_REQUEST_FAILED	BIL adjustment request failed.

17. Minimum Implementation Order

Build in this order:

1. Seed TCK technical-support categories.
2. Add ASR config migrations.
3. Seed diagnostic profiles.
4. Implement ASR guidance API.
5. Implement diagnostic capture API.
6. Implement known outage link API.
7. Patch TCK create-WO validation with ASR required diagnostic rules.
8. Implement WO event interpretation for technical trouble.
9. Implement repeat-fault detection.
10. Implement compensation recommendation.
11. Integrate BIL-02-ADJ-01 adjustment request.
12. Emit ASR business events.
13. Patch Backoffice screens.
14. Add E2E tests.

18. Test Scenarios

18.1 No Internet With WO

Input:

- NO_INTERNET ticket.
- Diagnostics captured.
- ONT offline.

Expected:

- Support WO can be created.
- Ticket becomes WAITING_WORK_ORDER.
- WO finalization resolves or reviews ticket.

18.2 Missing Diagnostics Blocks WO

Input:

- NO_INTERNET ticket.
- No diagnostic results.
- User attempts create WO.

Expected:

- API returns ASR_REQUIRED_DIAGNOSTIC_MISSING.
- No WO created.

18.3 Known Outage

Input:

- Outage check finds outage id.

Expected:

- Ticket linked to outage.
- Ticket routed to NOC/outage queue.
- No individual support WO by default.

18.4 Repeat Fault

Input:

- Same subscription had resolved NO_INTERNET ticket 10 days ago.

Expected:

- New ticket flagged or recategorized as REPEAT_FAULT.
- Priority/queue escalated by policy.

18.5 Compensation Recommendation

Input:

- WO confirms operator fault and service restored after outage threshold.

Expected:

- ASR compensation recommendation created.
- BIL adjustment request can be created.
- ASR does not directly credit wallet or invoice.

19. Developer Checklist

Before marking ASR-01 complete:

- No separate ASR ticket lifecycle table exists.
- Technical trouble uses TCK TECHNICAL_SUPPORT.
- Required diagnostics gate WO creation.
- TCK remains source of truth for ticket status.
- WO remains source of truth for field execution.
- Compensation calls BIL adjustment workflow, not wallet/invoice tables.
- Customer-safe summaries are separated from internal diagnostics.
- ASR events project to customer timeline.